

Introducción al lenguaje SPARQL

Ulises Cortés, Luis Oliva Felipe

{ia,loliva}@cs.upc.edu

<http://www.cs.upc.edu/~{ia,loliva}>

SID2017

<https://kemlg.upc.edu>



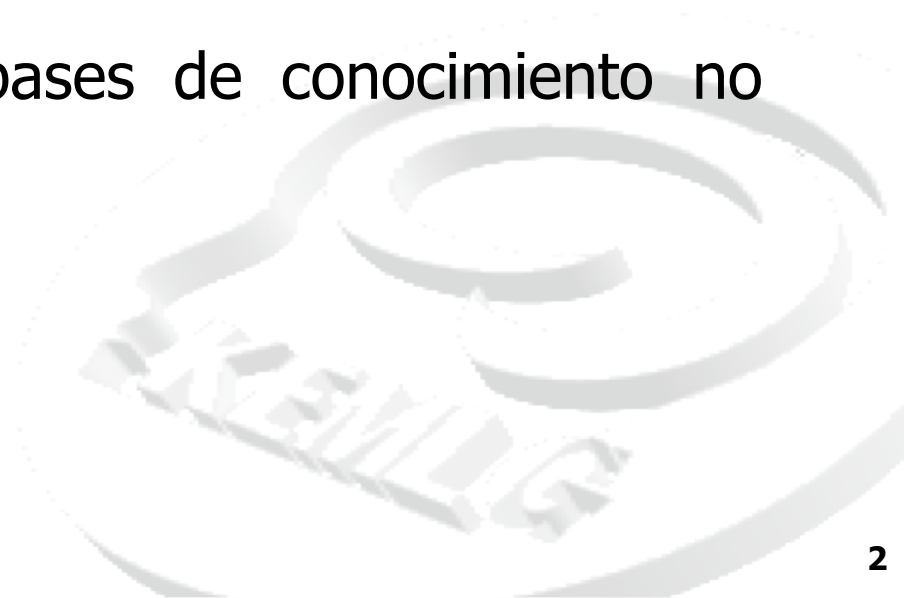
Knowledge Engineering and Machine Learning Group
UNIVERSITAT POLITÈCNICA DE CATALUNYA





Introducción

- SPARQL Protocol and RDF Query Language
- Lenguaje de consultas de-facto para la web semántica
 - Permite consultas en datos estructurados y semi-estructurados
 - Permite consultas sobre las estructuras de datos de forma natural
 - Permite *joins* sobre bases de conocimiento no homogéneas
- Versión 1.0 de 2008
 - Versión 1.1 de 2013





Introducción

- Se ejecuta contra datasets RDF
- Utilizaremos endpoints SPARQL para lanzar las queries
 - Genéricos, que accederán a datos RDF accesibles vía web
 - Específicos, que consultará datasets concretos
- Los resultados son
 - XML
 - JSON
 - RDF
 - HTML



Estructura de una consulta

- Prefijos: Permiten abreviar URIs que de otra forma serían muy largas (opcionales)
- Definición de modelos: Sobre que modelos RDF hacemos la consulta
- Resultado: Información que estamos obteniendo
- Patrón de consulta: Cómo obtenemos dicha información
- Modificadores de consulta: División, ordenado...

```
# prefix declarations
PREFIX foo: <http://example.com/resources/>
...
# dataset definition
FROM ...
# result
                                clause

SELECT ...
# query pattern
WHERE {
    ...
}
# query modifiers
ORDER BY ...
```



Sintaxis

- Basado en Turtle
 - URIs: <http://ejemplo.com/recurso> | prefijo:nombre
 - Literales: "string", "13.4"^^xsd:float (@en anotación lenguaje)
 - Tripletas: prefijo:nombre otroprefijo:propiedad "objeto" .
 - Variables: ?nombreVariable

- Ejemplo:

```
PREFIX foaf: <http://xmlns.com/foaf/0.1/>
SELECT ?name
FROM <http://dig.csail.mit.edu/2008/webdav/timbl/foaf.rdf>
WHERE { ?person foaf:name ?name . }
```



Ejemplo

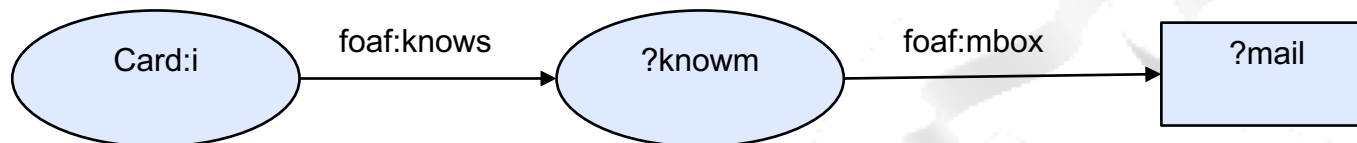
```
PREFIX foaf: <http://xmlns.com/foaf/0.1/>
```

```
PREFIX card: <http://www.w3.org/People/Berners-Lee/card#>
```

```
SELECT ?mail
```

```
FROM <http://dig.csail.mit.edu/2008/webdav/timbl/foaf.rdf>
```

```
WHERE {  
    card:i foaf:knows ?known .  
    ?known foaf:mbox ?mail . }  
}
```





Ejemplo de consulta I

```
PREFIX dbpedia: <http://dbpedia.org/property/birthName>
PREFIX actor: <http://dbpedia.org/ontology/Actor>
SELECT *
WHERE {
    ?actor a <http://dbpedia.org/ontology/Actor>.
    ?actor <http://dbpedia.org/property/birthName> ?nombre
} LIMIT 50
```

- Al poner * recuperamos todas las ?variables
- . Al final de cada línea en el WHERE
- Ojo: WHERE va entre corchetes, no paréntesis
- Uso de tripletas (notad como saltamos entre nodos)
- a es un alias para rdf:type

Ejemplo de consulta II

```
PREFIX actor: <http://dbpedia.org/ontology/Actor>
SELECT ?nombreActor, ?nombrePeli
WHERE {
    ?actor a <http://dbpedia.org/ontology/Actor>.
    ?peli <http://dbpedia.org/ontology/starring> ?actor.
    ?actor <http://dbpedia.org/property/name> ?nombreActor.
    ?peli <http://xmlns.com/foaf/0.1/name> ?nombrePeli.
} LIMIT 50
```

```
PREFIX actor: <http://dbpedia.org/ontology/Actor>
SELECT ?nombreActor, ?nombrePeli
WHERE {
    ?actor a <http://dbpedia.org/ontology/Actor>.
    ?peli <http://dbpedia.org/ontology/starring> ?actor.
    ?actor <http://dbpedia.org/property/name> ?nombreActor.
    ?peli <http://xmlns.com/foaf/0.1/name> ?nombrePeli.
    ?actor <http://dbpedia.org/property/name> "Bruce Lee"@en.
} LIMIT 50
```

```
PREFIX actor: <http://dbpedia.org/ontology/Actor>
SELECT ?nombreActor, ?nombrePeli
WHERE {
    ?actor a <http://dbpedia.org/ontology/Actor>.
    ?peli <http://dbpedia.org/ontology/starring> ?actor.
    ?actor <http://dbpedia.org/property/name> ?nombreActor.
    ?peli <http://xmlns.com/foaf/0.1/name> ?nombrePeli.
    FILTER regex(?nombreActor, "^Bruce")
} LIMIT 50
```



Modificadores de consulta útiles

- Limit
- Order by
 - Order by ?X desc ?Y
- Offset (para paginar)
- Distinct
- a (rdf:type)
- Construct (Grafo RDF como resultado)
- Optional (Bases heterogeneas)
- SameTerm (?X, ?Y)
- Podemos hacer subqueries incluyendo consultas dentro del where

Ejemplo Filter

PREFIX rdfs: <http://www.w3.org/2000/01/rdf-schema#>

PREFIX type: <http://dbpedia.org/class/yago/>

PREFIX prop: <http://dbpedia.org/property/>

prefix dbo: <http://dbpedia.org/ontology/>

SELECT ?country_name ?population

WHERE { ?country a type:WikicatLandlockedCountries ;

 rdfs:label ?country_name ;

 dbo:populationTotal ?population .

FILTER (?population > 15000000) .}

- También se puede filtrar mediante expresiones regulares
FILTER (regex(?variable, "RegExpr"))
- Se puede combinar con la anotación de idioma para filtrar las traducciones
FILTER langMatches(lang(?country_name), "FR") .

FILTER (?date > "1968-10-01"^^xsd:date && ?date < "1968-10-30"^^xsd:date)

Funciones internas para filtrar

- *Lógicas:* !, &&, ||
- *Matemáticas:* +, -, *, /
- *Comparación:* =, !=, >, <, IN, NOT IN...
- *SPARQL tests:* isURI, isBlank, isLiteral, isNumeric, bound
- str, lang, datatype
- *Other:* sameTerm, langMatches, regex, REPLACE
- *Condicionales (SPARQL 1.1):* IF, EXISTS, NOT EXISTS
- *Strings (SPARQL 1.1)*
STRLEN, SUBSTR, UCASE, LCASE, STRSTARTS, STRENDS, CONTAINS, STRBEFORE, STRAFTER, CONCAT



OPTIONAL

- A veces la información no es completa
- Optional permite intentar encontrar la información que le indiquemos sin hacer que toda la consulta falle

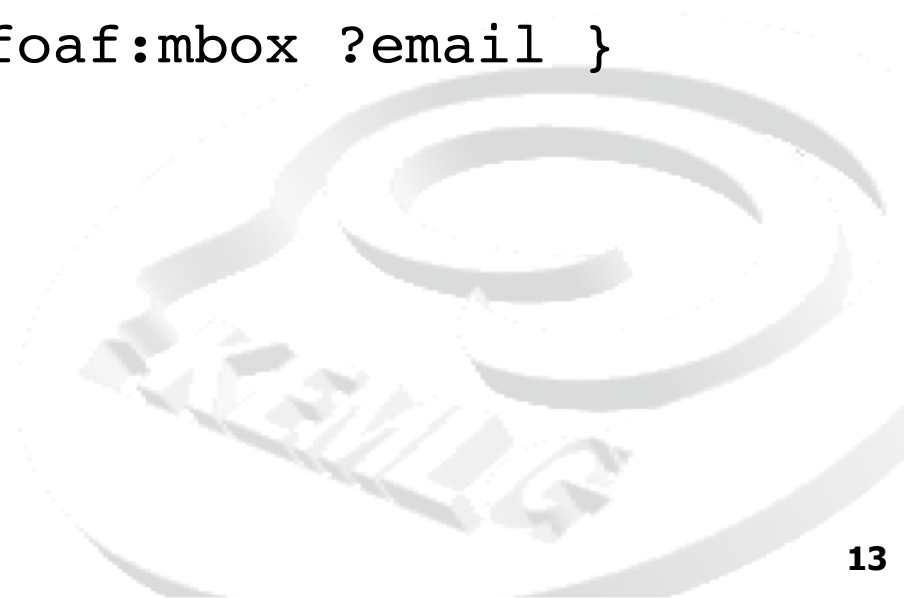
```
PREFIX vCard: <http://www.w3.org/2001/vcard-rdf/3.0#>  
PREFIX foaf: <http://xmlns.com/foaf/0.1/>
```

```
SELECT ?name ?url ?title  
FROM <http://www.w3.org/People/Berners-Lee/card>  
WHERE {  
  OPTIONAL { ?X foaf:name ?name . FILTER isLiteral(?name) . }  
  OPTIONAL { ?X foaf:homepage ?url . FILTER isURI(?url) . }  
  OPTIONAL { ?X foaf:title ?title . FILTER isLiteral(?title) . }  
}
```



Otro ejemplo con optional

```
PREFIX foaf: <http://xmlns.com/foaf/0.1/>
SELECT ?name ?email
FROM <http://www.w3.org/People/Berners-Lee/card>
WHERE {
    ?person foaf:name ?name .
    OPTIONAL { ?person foaf:mbox ?email }
}
```





Modificadores útiles II

- UNION: El resultado incluirá lo que se encuentre de aplicar ambos patrones de búsqueda

```
PREFIX dbp:<http://dbpedia.org/property/>
PREFIX dbo:<http://dbpedia.org/ontology/>
SELECT DISTINCT ?actor
WHERE {?actor dbp:name ?nombreActor.
      {?actor a dbo:Actor.} UNION
      {?actor dbp:occupation "Actor"@en.}
}
```

Modificadores útiles III

- MINUS: Elimina conjuntos

```
PREFIX foaf: <http://xmlns.com/foaf/0.1/>
SELECT ?name
FROM <http://dig.csail.mit.edu/2008/webdav/timbl/foaf.rdf>
WHERE { ?person foaf:name ?name .
MINUS {?person foaf:name "TellMeFirst"}}
```



Modificadores útiles IV

- COUNT
- BIND

```
SELECT ?nombre ?precioFinal
WHERE {
    ?x precio ?precio .
    ?x descuento ?descuento .
    ?x nombre ?nombre .
    BIND (?precio*(1-?descuento) AS ?precioFinal) }
```



Ejercicio

- Descargar:
<http://dig.csail.mit.edu/2008/webdav/timbl/foaf.rdf>
- Encontrar la gente en la Vcard de Tim Berners-Lee que no tenga mail
- Recuperar la 2ª página de resultados de nombres de la Vcard de Tim Berners-Lee





Haciendo preguntas

- ¿Es más largo el Amazonas que el Nilo?

```
PREFIX prop: <http://dbpedia.org/property/>
```

```
ASK {
```

```
<http://dbpedia.org/resource/Amazon_River> prop:length ?amazon .
```

```
<http://dbpedia.org/resource/Nile> prop:length ?nile .
```

```
FILTER(?amazon > ?nile) .
```

```
}
```





DML

- **INSERT**

```
INSERT{<http://example/libro>titulo "Snowcrash";
      Autor "Neal Stephenson";
      Precio "Mucho"}
WHERE{}
```

- **DELETE**

```
DELETE{?x} WHERE{?x :titulo "SnowCrash"}
```

```
PREFIX foaf: <http://xmlns.com/foaf/0.1/>
```

```
WITH <http://example/addresses>
```

```
DELETE { ?person foaf:givenName 'Bill' }
```

```
INSERT { ?person foaf:givenName 'William' }
```

```
WHERE {?person foaf:givenName 'Bill'}
```

DML II

- UPDATE

```
INSERT DATA { <http://example/libro> PrecioReal "Mucho" }
```

```
DELETE DATA { <http://example/libro> PrecioReal "Mucho" }
```

- Otras

- Load
- Copy
- Move
- Clear





Ejercicios

- Probar las consultas en la DBPedia
 - <http://dbpedia.org/sparql>
- Buscar películas producidas por “Charlton Heston”
 - Productores:
<<http://dbpedia.org/class/yago/FilmMaker110088390>>
- Buscar nombres de actores porno (AdultActor) y películas en las que han participado
- Añadir el director a la búsqueda anterior
- Buscar nombres de actores españoles y películas en las que han participado
- Buscar actores que han ganado un Goya
- ¡No olvidéis limitar a 50 la búsqueda!



Ejercicios

- Usando el endpoint:
 - <http://dbtune.org/jamendo/store/user/query>
- Busca artistas musicales (MusicArtist) y da su nombre, su imagen, página web y localización cercana (si tienen).
- El concepto MusicArtist está definido en la Music Ontology (<http://purl.org/ontology/mo>)





Referencias

- <https://www.w3.org/TR/2013/REC-sparql11-query-20130321/>
- <https://www.w3.org/TR/2013/REC-sparql11-update-20130321/>
- <https://www.w3.org/wiki/SparqlEndpoints>





Knowledge Engineering and Machine Learning Group UNIVERSITAT POLITÈCNICA DE CATALUNYA

Ulises Cortés, Luis Oliva Felipe

{ia,loliva}@lsi.upc.edu

<http://www.lsi.upc.edu/{ia,loliva}>

2017

